Dispatchly Project Design

Intelligent Delivery Management and Verification Platform

September 4, 2026

Contents

1 Project Overview

Dispatchly is a multi-role delivery management platform designed to coordinate the complete lifecycle of delivery requests from order creation to successful delivery verification.

The system connects three primary operational actors:

- Retailers who create delivery requests.
- Dispatchers who manage delivery allocation and assign riders.
- Riders who collect, transport, and complete deliveries through QR-based verification.

The primary goal of the platform is to provide a reliable, traceable, and operationally efficient delivery workflow while maintaining data consistency and accountability throughout the delivery lifecycle.

2 System Objectives

The key objectives of Dispatchly are:

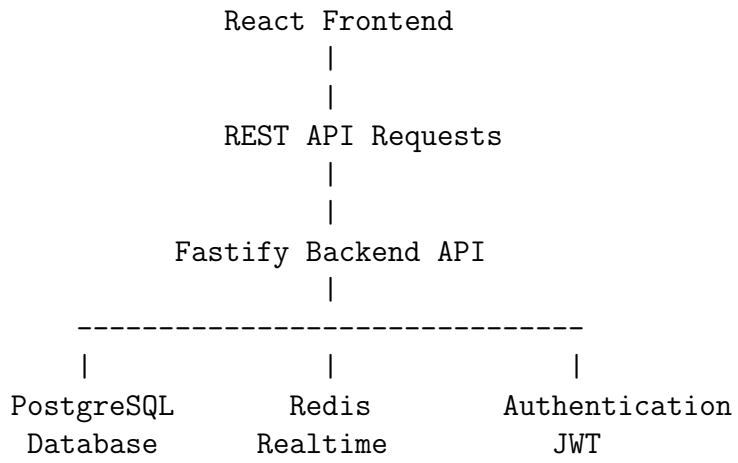
1. Provide retailers with a simple mechanism to create delivery requests.
2. Enable dispatchers to monitor deliveries and allocate riders.
3. Provide riders with an operational dashboard for assigned deliveries.
4. Ensure delivery custody transfer through QR verification.
5. Maintain an auditable history of delivery status changes.
6. Support real-time operational visibility.

3 System Architecture

Dispatchly follows a client-server architecture consisting of:

- React-based frontend application.
- Fastify and TypeScript backend API.
- PostgreSQL relational database.
- Redis-based real-time communication layer.

The high-level architecture is:



4 User Roles and Responsibilities

4.1 Retailer

Retailers initiate delivery requests.

Responsibilities:

- Create delivery orders.
- Provide customer details.
- Specify package information.
- Provide payment information.
- Display pickup QR code for rider verification.

4.2 Dispatcher

Dispatchers manage delivery operations.

Responsibilities:

- View pending deliveries.
- Monitor delivery statuses.
- Assign available riders.
- Coordinate delivery execution.

4.3 Rider

Riders execute physical delivery operations.

Responsibilities:

- View assigned deliveries.

- Verify pickup using QR scanning.
- Transport deliveries.
- Verify final delivery using customer QR scanning.

5 Delivery Lifecycle Design

The delivery workflow follows a controlled state transition model.

```
Pending
   |
   |
Dispatcher Assignment
   |
   v
Assigned
   |
   |
Pickup QR Verification
   |
   v
In Transit
   |
   |
Delivery QR Verification
   |
   v
Delivered
```

The system prevents unauthorized manual status changes. Critical state transitions are triggered through verified operational events.

6 Backend Design

The backend is implemented using:

- Node.js
- Fastify framework
- TypeScript
- PostgreSQL

The backend follows a modular route-based architecture.

6.1 Authentication Module

Authentication is implemented using JWT tokens.

Each request requiring authorization validates:

- User identity.
- User role.
- Business ownership.

6.2 Delivery Request Module

The delivery request module manages order creation.

A delivery request contains:

- Customer information.
- Delivery address.
- Package description.
- Payment details.
- Current delivery status.
- Version number for concurrency control.

7 Dispatcher Assignment Workflow

Dispatcher assignment is implemented through:

```
POST /delivery-requests/:id/assign
```

The assignment process:

1. Dispatcher selects a pending delivery.
2. Dispatcher selects an active rider.
3. Backend validates the request.
4. Assignment record is created.
5. Delivery status changes from pending to assigned.
6. Rider dashboard receives the updated delivery.

8 Rider Workflow Design

The rider workflow is based on operational verification rather than manual status updates.

8.1 Assigned Deliveries

Riders retrieve deliveries using:

```
GET /riders/me/deliveries
```

The response contains:

- Customer information.
- Package information.
- Delivery status.
- Delivery version.

8.2 Pickup Verification

Pickup verification ensures that custody is transferred correctly.
Process:

```
Retailer displays Pickup QR

          |

Rider scans QR code

          |

POST /delivery-requests/:id/pickup

          |

Status becomes IN_TRANSIT
```

8.3 Delivery Verification

Final delivery completion follows:

```
Customer displays Delivery QR

          |

Rider scans QR code

          |
```

```
POST /delivery-requests/:id/delivery

          |

Status becomes DELIVERED
```

9 QR Verification Design

QR verification provides:

- Proof of custody transfer.
- Prevention of unauthorized delivery completion.
- Operational accountability.

The QR scanner implementation supports:

- Mobile camera access.
- Environment-facing camera selection.
- Automatic scanning.
- Scanner cleanup after completion.

10 Database Design

The primary database entities include:

10.1 Users

Stores:

- Retailers.
- Dispatchers.
- Riders.

10.2 Delivery Requests

Stores:

- Delivery details.
- Current status.
- Payment information.
- QR information.

10.3 Assignments

Maintains the relationship between:

- Delivery requests.
- Riders.

10.4 Status Events

Provides an audit trail of delivery changes.
Example:

```
pending
    |
assigned
    |
in_transit
    |
delivered
```

11 Concurrency Control

Dispatchly uses optimistic concurrency control through delivery version numbers.
Each update operation verifies the current version before applying changes.
Example:

```
Client Version = 3

Database Version = 3

Update accepted


Client Version = 3

Database Version = 4

Conflict returned
```

12 Frontend Design

The frontend is implemented using React and TypeScript.
Major frontend modules:

- Authentication pages.
- Retailer dashboard.
- Dispatcher dashboard.
- Rider dashboard.
- QR scanner component.

13 Dispatcher Dashboard Design

The dispatcher interface focuses on operational management.

Features:

- Delivery overview.
- Status monitoring.
- Rider assignment.
- Real-time updates.

14 Rider Dashboard Design

The rider dashboard is designed for operational efficiency.

The dashboard provides:

- Assigned delivery visibility.
- Current delivery status.
- Pickup verification actions.
- Delivery completion actions.

The interface is designed to support large delivery volumes through table-based operational views rather than individual delivery cards.

15 Real-Time Communication

Redis is used to support real-time delivery updates.

Events are triggered when:

- Delivery status changes.
- Rider assignments occur.
- Verification events complete.

16 Security Considerations

Security mechanisms include:

- JWT authentication.
- Role-based access control.
- Business-level data isolation.
- QR verification.
- Validation using schema-based checking.

17 Future Improvements

Future development areas include:

- Rider route optimization.
- Delivery location tracking.
- Automated rider allocation.
- Advanced analytics dashboard.
- Customer notifications.

18 Conclusion

Dispatchly provides a structured delivery management architecture that connects retailers, dispatchers, and riders through a secure operational workflow.

The implemented system emphasizes accountability, traceability, and reliable delivery execution through role-based dashboards, QR verification, and controlled status transitions.

The current architecture provides a strong foundation for scaling delivery operations while maintaining operational visibility and data integrity.